VPA A-Level CS — AS Level Semester 2 Course Overview

VPA A-Level Computer Science — AS Level Semester 2

Course Overview: Programming Intensive & Consolidation 编程强化与巩固

Course Code: CIE A-Level Computer Science (9618)

Semester Duration: 18 weeks (February – June)

Lesson Duration: 40 minutes

Target Students: Year 11 equivalent, EAL (Mandarin L1), mixed ability

Papers Covered: Paper 1 consolidation + Paper 2 (Fundamental Problem-solving & Programming, 2h, 75 marks)

1. Course Summary and Philosophy

This semester is the programming heart of the AS Level. Students will develop fluency in algorithm design, data structures, and programming constructs using CIE pseudocode. Building on the theoretical foundations from Semester 1, students now apply computational thinking to solve problems systematically. The semester also consolidates Paper 1 knowledge through integrated revision and mock examinations.

本学期是AS水平的编程核心。学生将使用CIE伪代码培养算法设计、数据结构和编程结构的流利度。

2. Prerequisite Knowledge from Semester 1

Prerequisite	Source	Application in Semester 2
Binary/hex conversions	Weeks 1–2	Understanding memory addresses, bitwise operations
Two's complement	Week 1	Integer representation in programming
CPU architecture	Week 7	Understanding how code executes
Assembly language	Weeks 10–11	Low-level thinking informs high-level code
System software	Week 12	Understanding compilers, runtime environments
Pseudocode basics	Week 17	Foundation for complex algorithms

3. Eighteen-Week Curriculum Map

Week	Topic (Syllabus Ref)	Paper	Key Content	Assessment
1	2.1 Algorithm Design — Pseudocode Mastery	P2	Variables, constants, I/O, expressions	Baseline programming test
2	2.1 — Selection & Iteration	P2	IF/CASE, FOR/WHILE/REPEAT	Code tracing quiz

Week	Topic (Syllabus Ref)	Paper	Key Content	Assessment
3	2.1 — Nested Structures & Logic	P2	Nested IF, compound conditions, flags	Programming task
4	2.3 — Subroutines (Procedures & Functions)	P2	Parameter passing, return values, scope	Practical test
5	2.3 — Recursion	P2	Base case, recursive case, stack frames	Recursion worksheet
6	2.3 — String & File Handling	P2	String operations, text file read/write	Mid-unit test
7	2.2 — 1D Arrays	P2	Declaration, traversal, search, sort	Practical task
8	2.2 — 2D Arrays & Records	P2	Tables, matrices, user-defined types	Programming task
9	Consolidation & Half-Term Assessment	P2	Review weeks 1–8; Paper 2 Section A	Half-Term Exam
10	2.2 — Files & CSV Processing	P2	Sequential files, CSV parsing, merging	Data processing task
11	2.4 — Testing & Debugging	P2	Test plans, trace tables, breakpoints	Test plan exercise
12	2.4 — Error Types & Handling	P2	Syntax, logic, runtime; validation	Error identification
13	2.4 — Software Development Life Cycle	P2	Waterfall, agile, prototyping	Research project
14	2.4 — Documentation & Maintenance	P2	Technical docs, user guides, version control	Documentation task
15	Paper 1 Revision — Topics 1.1–1.4	P1	Information representation, hardware	Revision test
16	Paper 1 Revision — Topics 1.5–1.7	P1	System software, security, ethics	Revision test
17	Mock Examination Series	P1/P2	Full Paper 1 + Paper 2 practice	Mock Exams
18	Final Review & Summer Preparation	P1/P2	Gap analysis, individual targets	Target setting

4. Programming Language: CIE Pseudocode

Why Pseudocode (not Python)?

CIE Paper 2 requires answers in pseudocode. Students must be fluent in CIE conventions. However, Python is used for practical demonstrations and verification.

为什么使用伪代码？CIE试卷2要求用伪代码作答。学生必须熟悉CIE规范。Python用于实践演示和验证。

CIE Pseudocode Quick Reference

Construct	Syntax	Example
Variable declaration	DECLARE <name> : <type>	DECLARE Score : INTEGER
Constant	CONSTANT <name> = <value>	CONSTANT Pi = 3.14159
Assignment	<variable> ← <expression>	Score ← 100
Input	INPUT <variable>	INPUT Name
Output	OUTPUT <expression>	OUTPUT "Hello", Name
IF...THEN	IF <condition> THEN ... ENDIF	IF Score > 50 THEN OUTPUT "Pass" ENDIF
IF...ELSE	IF ... THEN ... ELSE ... ENDIF	IF Score >= 50 THEN OUTPUT "Pass" ELSE OUTPUT "Fail" ENDIF
Nested IF	IF ... THEN IF ... THEN ... ENDIF ENDIF	Grade boundaries
CASE	CASE <expr> OF ... ENDCASE	CASE Grade OF "A": OUTPUT "Excellent" ENDCASE
FOR loop	FOR <var> = <start> TO <end> ... ENDFOR	FOR i = 1 TO 10 OUTPUT i ENDFOR
FOR with STEP	FOR ... TO ... STEP ...	FOR i = 10 TO 1 STEP -1
WHILE loop	WHILE <condition> DO ... ENDWHILE	WHILE Count < 5 DO ... ENDWHILE
REPEAT loop	REPEAT ... UNTIL <condition>	REPEAT ... UNTIL Choice = "Q"
Procedure	PROCEDURE <name>(<params>) ... ENDPROCEDURE	PROCEDURE Greet(Name) ... ENDPROCEDURE
Function	FUNCTION <name>(...) RETURNS <type> ... ENDFUNCTION	FUNCTION Max(a, b) RETURNS INTEGER ... ENDFUNCTION
Call procedure	CALL <name>(<args>)	CALL Greet("Alice")
Call function	<variable> ← <name>(<args>)	Result ← Max(5, 3)
1D Array	DECLARE <name> : ARRAY[<l>:<u>] OF <type>	DECLARE Marks : ARRAY[1:5] OF INTEGER
2D Array	DECLARE <name> : ARRAY[<l1>:<u1>, <l2>:<u2>] OF <type>	DECLARE Grid : ARRAY[1:3, 1:3] OF INTEGER
Record/Type	TYPE <name> = ... ENDTYPE	TYPE Student = Name: STRING, Age: INTEGER ENDTYPE
File open	OPENFILE <file> FOR <mode>	OPENFILE "data.txt" FOR READ
File read	READFILE <file>, <variable>	READFILE "data.txt", Line
File write	WRITEFILE <file>, <data>	WRITEFILE "out.txt", Result
File close	CLOSEFILE <file>	CLOSEFILE "data.txt"
String length	LENGTH(<string>)	Len ← LENGTH("Hello")
Substring	<string>[<start>:<end>]	First ← Text[1:3]
String to integer	INT(<string>)	Num ← INT("42")
Integer to string	STR(<number>)	Text ← STR(42)

Construct	Syntax	Example
ASCII code	ASC(<char>)	Code ← ASC("A")
Character from ASCII	CHR(<code>)	Char ← CHR(65)
Random number	RANDOM() or RANDOMBETWEEN(a, b)	Dice ← RANDOMBETWEEN(1, 6)
MOD	<a> MOD <b>	Remainder ← 17 MOD 5
DIV	<a> DIV <b>	Quotient ← 17 DIV 5

5. Standard Programming Lesson Structure (40 Minutes)

0 – 5 min: Code Review / Do Now

☐ Review previous lesson's code on screen

☐ Spot the error / predict the output challenge

☐ Vocabulary check (bilingual flashcards)

5 – 10 min: Concept Introduction

☐ New construct explained with visual diagram

☐ Live coding demonstration (teacher projects screen)

☐ Common errors highlighted upfront

10 – 25 min: Guided & Independent Practice

☐ Bronze task: Complete partially written code

☐ Silver task: Write code from specification

☐ Gold task: Extend with additional features / optimisation

☐ Teacher circulates, gives immediate feedback

25 – 35 min: Code Review & Debugging

☐ Students share solutions (volunteer or random selection)

☐ Class identifies bugs and improvements

☐ Efficiency discussion (e.g., "Can we do this with fewer lines?")

35 – 40 min: Set Programming Homework

☐ Extend class task

☐ Code reading exercise (trace table)

☐ Research: How is this construct used in real software?

6. Differentiation Framework (Programming)

Tier	Task Modification	Support	Extension
Bronze	Complete scaffolded code; fill blanks; modify existing code	Step-by-step guide; syntax reference card; paired programming	Add simple validation; basic output formatting
Silver	Write code from specification with hints	Partial flowchart provided; keyword bank; debugging checklist	Refactor using subroutines; add user-friendly features

Tier	Task Modification	Support	Extension
Gold	Design and implement from problem description only	Pseudocode self-generated; independent research encouraged	Optimise for efficiency; handle edge cases; implement alternate algorithms

7. EAL Support for Programming

Programming-Specific EAL Strategies:

☐ Bilingual keyword wall: 变量 (variable), 循环 (loop), 条件 (condition), 函数 (function), 数组 (array)

☐ Code comments in Chinese allowed during drafting (must translate to English for submission)

☐ Visual syntax cards: Colour-coded pseudocode blocks (green=input, blue=processing, red=output, yellow=control)

☐ Cognate highlighting: "function" = 函数 (hánshù), "variable" = 变量 (biànliàng), "parameter" = 参数 (cānshù)

☐ Sentence frames for explaining code: "This loop repeats while..." / "This function returns..."

☐ Pair programming: EAL student with stronger English speaker; roles rotate

☐ Pre-teach abstract concepts with concrete analogies: "Array = row of lockers, each with number"

8. Key Bilingual Glossary (Programming Focus)

Algorithm 算法 (suànfǎ) — Step-by-step problem-solving procedure

Pseudocode 伪代码 (wěi dàimǎ) — Language-neutral code description

Variable 变量 (biànliàng) — Named storage location for data

Constant 常量 (chángliàng) — Fixed value that doesn't change

Data type 数据类型 (shùjù lèixíng) — Category of data (INTEGER, STRING, etc.)

Operator 运算符 (yùnsuàn fú) — Symbol for operation (+, -, AND, OR)

Selection 选择 (xuǎnzé) — Decision structure (IF, CASE)

Iteration 迭代/循环 (diédài/xúnhuán) — Repetition structure (FOR, WHILE, REPEAT)

Subroutine 子程序 (zǐchéngxù) — Reusable code block (procedure/function)

Parameter 参数 (cānshù) — Input value passed to subroutine

Return value 返回值 (fǎnhuí zhí) — Output from a function

Local variable 局部变量 (júbù biànliàng) — Variable only accessible within subroutine

Global variable 全局变量 (quánjú biànliàng) — Variable accessible throughout program

Recursion 递归 (dìguī) — Function calling itself

Base case 基准情况 (jīzhǔn qíngkuàng) — Condition that stops recursion

Array 数组 (shùzǔ) — Collection of elements with same type

Index 索引 (suǒyǐn) — Position identifier in array

Record 记录 (jìlù) — Composite data type with multiple fields

File 文件 (wénjiàn) — Persistent storage of data

CSV 逗号分隔值 (dòuhào fēngé zhí) — Comma-separated values format

Syntax error 语法错误 (yǔfǎ cuòwù) — Code breaking language rules

Logic error 逻辑错误 (luójí cuòwù) — Code runs but produces wrong result

Runtime error 运行时错误 (yùnxíng shí cuòwù) — Error during execution

Test data 测试数据 (cèshì shùjù) — Input used to verify program

Validation 验证 (yànzhèng) — Checking input meets criteria

Verification 核对 (héduì) — Checking data entry accuracy

Trace table 追踪表 (zhuīzōng biǎo) — Table tracking variable values

Breakpoint 断点 (duàndiǎn) — Pause point for debugging

Waterfall model 瀑布模型 (pùbù móxíng) — Linear SDLC approach

Agile 敏捷 (mǐnjié) — Iterative SDLC approach

9. Resource Requirements

Resource	Purpose	Frequency
CIE 9618 pseudocode specification	Syntax authority	Every lesson
Online Python interpreter (repl.it, Google Colab)	Code verification	Weekly
Past Paper 2 questions (9618)	Exam practice	Weekly from Week 4
Trace table templates	Debugging practice	Every 2 weeks
Sample CSV datasets	File handling practice	Weeks 10–11
Version control introduction (Git basics)	SDLC realism	Week 14
Bilingual programming dictionary	EAL support	Every lesson

10. Past Paper Reference Guide (Paper 2)

Topic	Recommended Past Papers	Question Types
Pseudocode fundamentals	9618/21 MJ22 Q1, 9618/22 ON21 Q1	Trace tables, code completion
Selection & iteration	9618/21 MJ23 Q2, 9618/22 ON22 Q2	Write code, identify constructs
Subroutines	9618/21 ON22 Q3, 9618/22 MJ22 Q3	Parameter passing, scope
Recursion	9618/21 MJ22 Q4, 9618/22 ON21 Q4	Trace recursive calls
Arrays & records	9618/21 MJ23 Q3, 9618/22 ON22 Q4	Declaration, traversal, search
Files & CSV	9618/21 ON22 Q5, 9618/22 MJ23 Q5	Read/write operations
Testing & debugging	9618/21 MJ23 Q6, 9618/22 ON22 Q6	Test plans, error identification
SDLC & documentation	9618/21 ON22 Q6, 9618/22 MJ22 Q6	Essay questions, comparisons

Victoria Park Academy Shenzhen — A-Level Computer Science Curriculum 2024–2026